

This is CS50

Week 2

Kelly Ding (she/her)

Preceptor

kelly@cs50.harvard.edu

- What are the steps involved in **compilation**?
- When should we use **arrays**?
- What are **strings**, really?
- What's the point of **command-line arguments**?

- What are the steps involved in **compilation**?
- When should we use **arrays**?
- What are **strings**, really?
- What's the point of **command-line arguments**?
- What makes for good **design**?

Part 0

Design

What are the
characteristics of
good design?

Is your code efficient?

Avoid unnecessary **repetition** and **redundant operations**.

- Avoiding copy-pasting code, even with slight changes.
Instead, consider loops or abstraction via functions.
- If a value is computed multiple times, consider storing in a variable to prevent redundant computation.

Is your code easy to maintain?

- **Prototype** functions at the top of their code file, above `main()`.
- Avoid “**magic numbers.**” (i.e., hardcoded numbers.) If a value will not change throughout the program, it should be defined as a **constant**.

Which is better and why?

```
1 // Option A
2 for (int i = 0; i < strlen(string); i++)
3
4 // Option B
5 for (int i = 0, n = strlen(string); i < n; i++)
```

Which is better and why?

```
1 // Option A
2 const int MONTHS_IN_YEAR = 12;
3 for (int i = 0; i < MONTHS_IN_YEAR; i++)
4
5 // Option B
6 for (int i = 0; i < 12; i++)
```

Which is better and why?

```
1 // Option A
2 int main(void)
3 {
4     if (EXPRESSION)
5     {
6         return 0;
7     }
8     return 1;
9 }
```

```
1 // Option B
2 int main(void)
3 {
4     if (EXPRESSION)
5     {
6         return 0;
7     }
8     else
9     {
10        return 1;
11    }
12 }
```

Which is better and why?

```
1 // Option A
2 int main(void)
3 {
4     bool success = true;
5     if (success)
6     {
7         // Lots of code
8         return 0;
9     }
10    else
11    {
12        return 1;
13    }
14 }
```

```
1 // Option B
2 int main(void)
3 {
4     bool success = true;
5     if (!success)
6     {
7         return 1;
8     }
9     // Lots of code
10    return 0;
11 }
```

Questions?



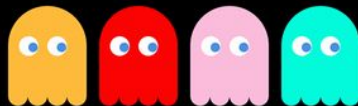
Part 1

Arrays

LEADERBOARD

RANK	NAME	SCORE
------	------	-------

1	JENNA	98 765
2	TOM	92 000
3	JENNA	91 535
4	GRACE	89 125
5	PHIL	88 910



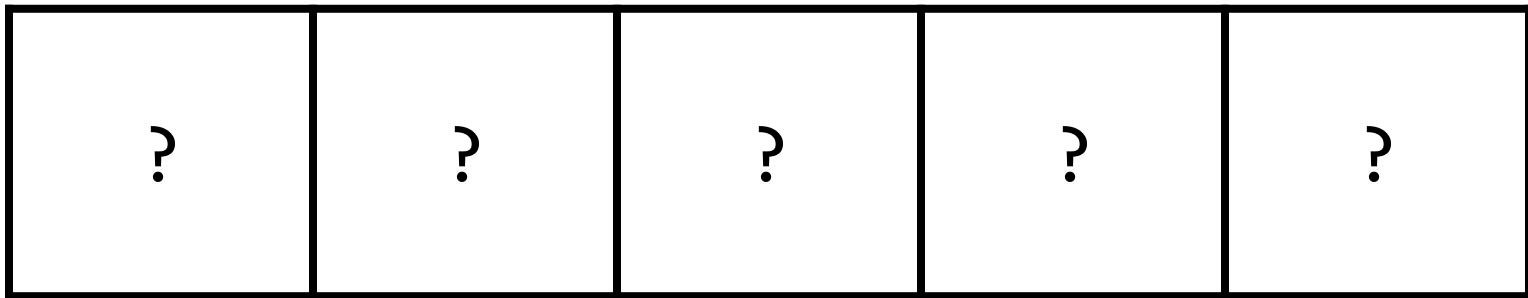
PRESS ENTER
TO PLAY AGAIN

scores

120	86	56	42	23
-----	----	----	----	----


```
int scores[5];
```

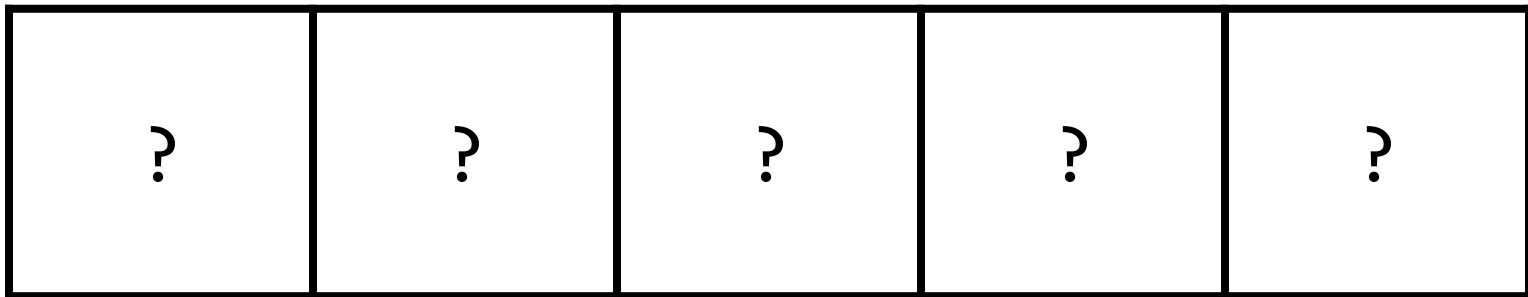
scores



name

```
int scores[5];
```

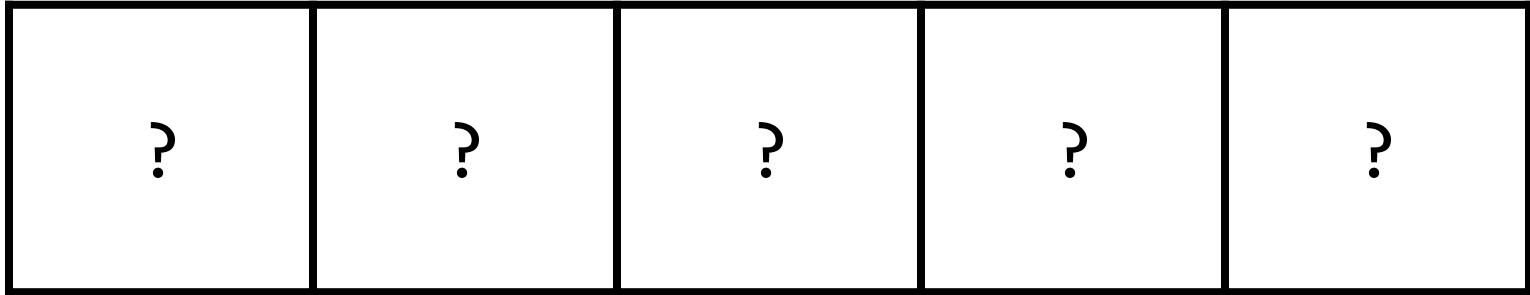
scores



size

```
int scores[5];
```

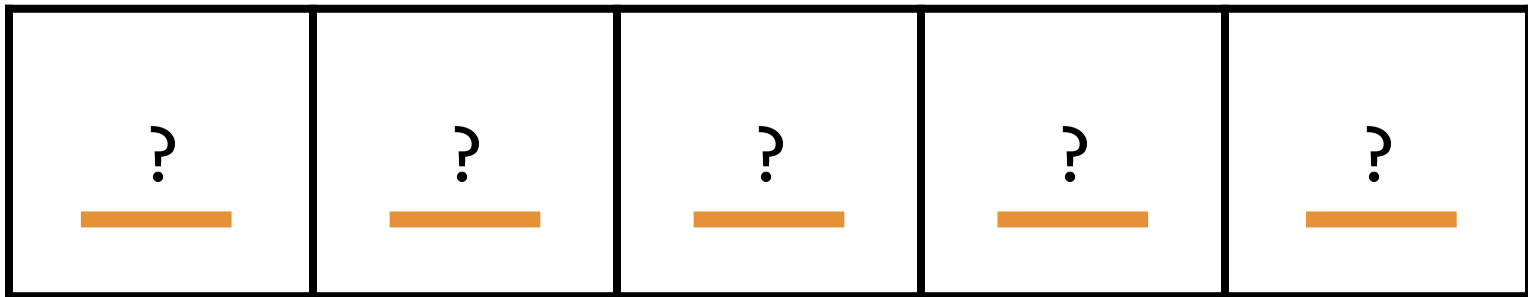
scores



type

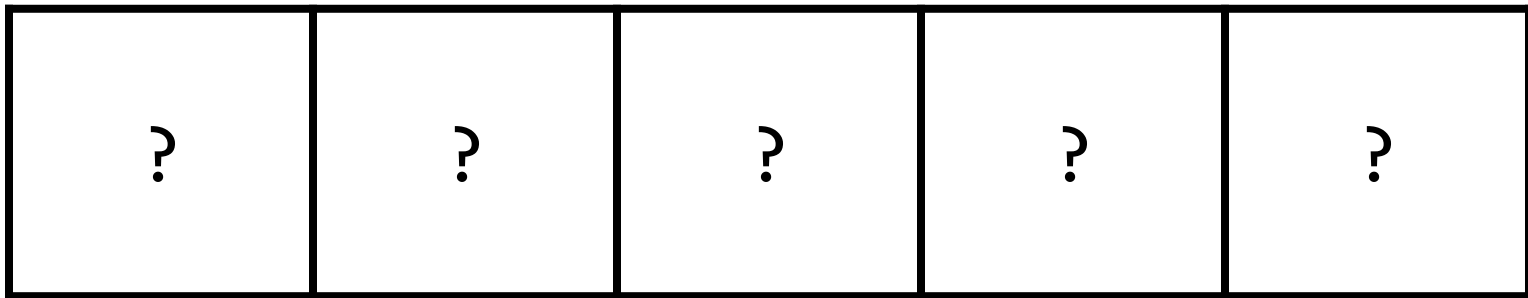
int scores[5];

scores



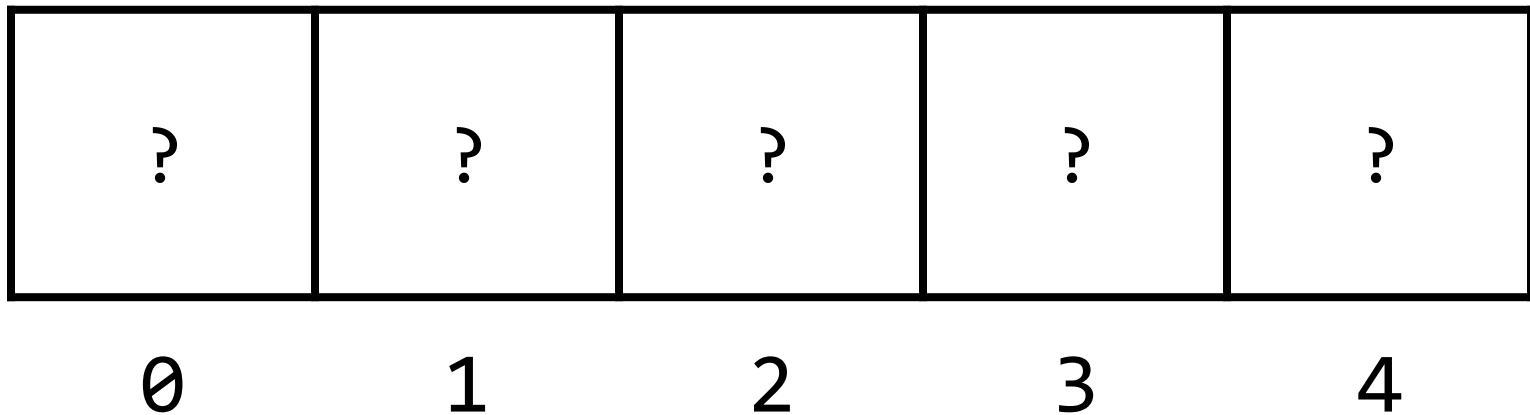
```
int scores[5];
```

scores



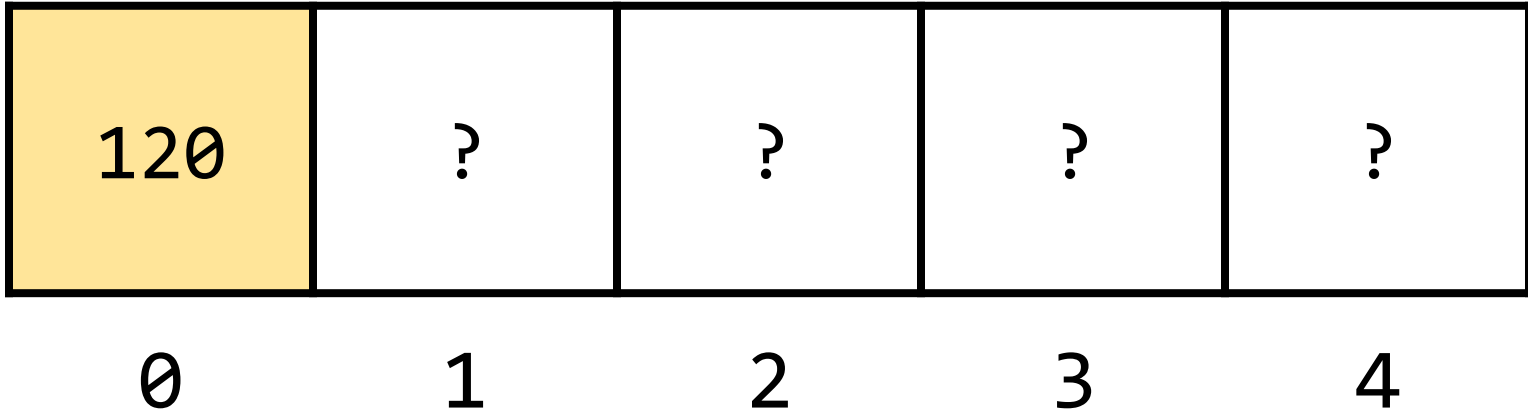
```
int scores[5];
```

scores



```
int scores[5];  
scores[0] = 120;
```

scores



```
int scores[5];  
scores[0] = 120;  
scores[1] = 86;
```

scores

120	86	?	?	?
0	1	2	3	4


```
int scores[5] = {120, 86, 56, 42, 23};
```

scores

120	86	56	42	23
-----	----	----	----	----

0

1

2

3

4

```
int scores[5] = {120, 86, 56, 42, 23};  
  
for (int i = 0; i < 5; i++)  
{  
    printf("%i\n", scores[i]);  
}
```

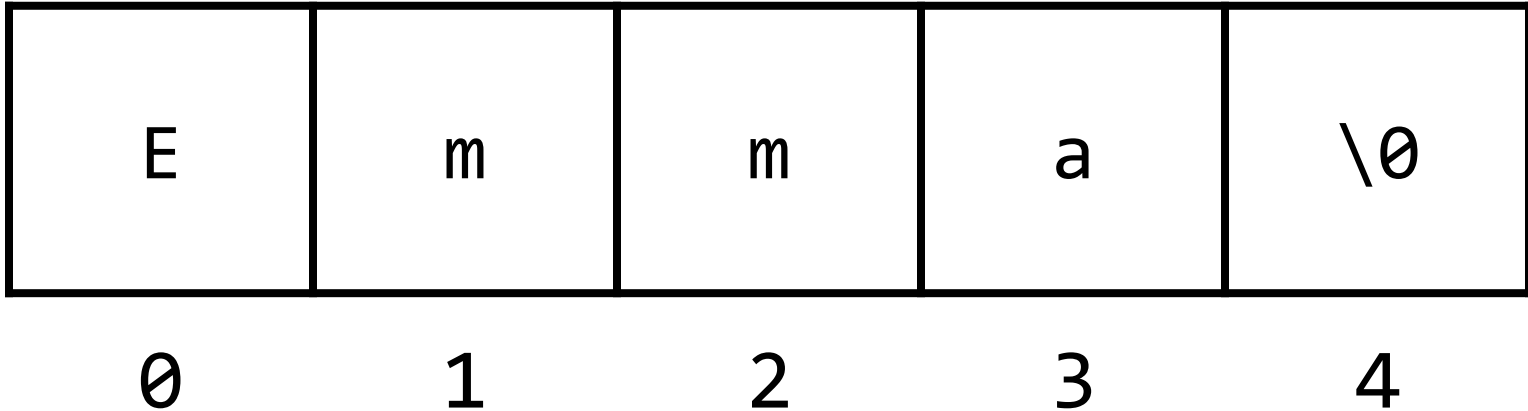
```
int scores[5] = {120, 86, 56, 42, 23};  
  
for (int i = 0; i < 5; i++)  
{  
    printf("%i\n", scores[i]);  
}
```

Part 2

Strings

```
string name = "Emma";
```

name



name[0];

name

E	m	m	a	\0
---	---	---	---	----

0

1

2

3

4

name[1];

name

E	m	m	a	\0
0	1	2	3	4

Questions?



Character-by-Character Exercise

Write a program, `print.c`, that takes a string as input and prints it character-by-character.

```
$ ./print  
Input: Hello  
Hello
```

Reverse Exercise

Write a program, `reverse.c`, that takes a string as input, and reverses it.

```
$ ./reverse  
Text: This is CS50.  
Reverse: .05SC si sihT
```

Palindrome Exercise

Write a program, `palindrome.c`, that takes a string as input, and determines whether it is a palindrome (the same backwards and forwards).

```
$ ./palindrome
Text: racecar
Palindrome!
$ ./palindrome
Text: jellyfish
Not palindrome.
```

Initials Exercise

Write a program, `initials.c`, that takes a user's full name as input using `get_string`, and outputs their initials.

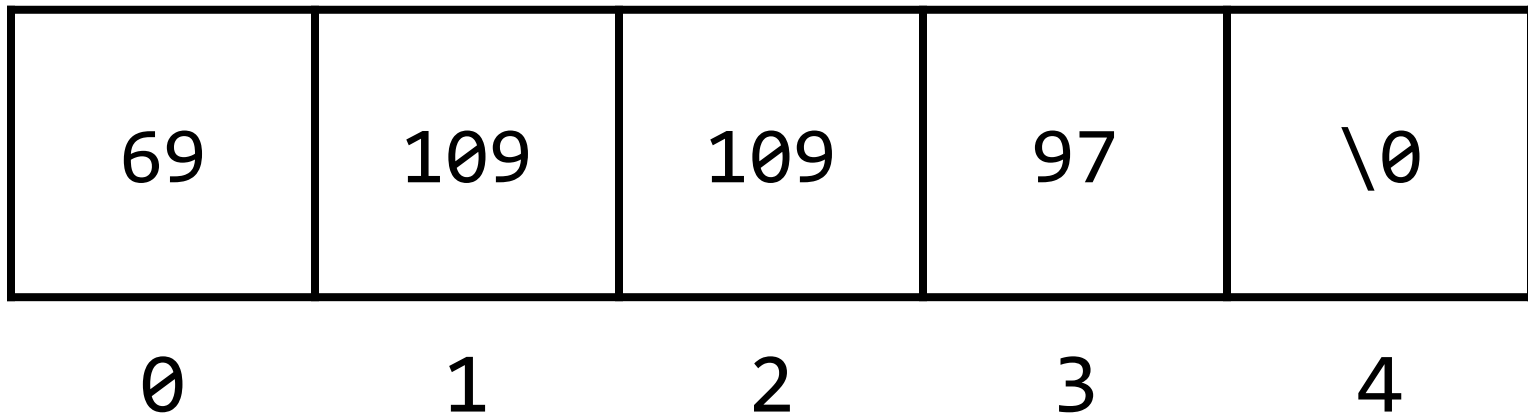
```
$ ./initials  
Name: David J. Malan  
DJM
```

A	B	C	...	Z
65	66	67	...	90

a	b	c	...	z
97	98	99	...	122

```
string name = "Emma";
```

name



Alphabetical Exercise

Write a program, `alphabetical.c`, that prints “Yes” if a lowercase string’s characters are in alphabetical order, and “No” if they are not.

```
$ ./alphabetical
Input: abcdefg
Yes
$ ./alphabetical
Input: gfedcba
No
```

Anagram Exercise

Write a program, `anagram.c`, that takes two words as input, and determines whether the words are anagrams.

- An anagram is a word, phrase, or name formed by rearranging the letters of another, such as *cinema*, formed from *iceman*.
- If two words are an exact match, output “Exact match.”
- If either word contains non-alphabetic characters, output “Alphabetic characters only” and return 1.

```
$ ./anagram
Word 1: listen
Word 2: silent
Anagram!
```

```
$ ./anagram
Word 1: program
Word 2: silent
Not anagram.
```

```
$ ./anagram
Word 1: silent
Word 2: silent
Exact match.
```

```
$ ./anagram
Word 1: this
Word 2: cs50
Alphabetic characters only.
```


Part 3

Command-Line Arguments

What are some examples of programs we've seen that take command-line arguments?

\$ make mario

```
$ ./caesar 13
```

```
int calculate_quarters(int cents)
{
    ...
}
```

Function argument(s)



```
int calculate_quarters(int cents)
{
    ...
}
```

Return type



```
int calculate_quarters(int cents)
{
    ...
}
```

```
int main(void)
{
    ...
}
```



```
int main(int argc, string argv[])  
{  
    ...  
}
```

Why use command-line arguments instead of standard input (`get_int` and `get_string`)?

\$ make mario

argv

"make"	"mario"
--------	---------

argc

2

\$./caesar 13

argv

"./caesar"	"13"
------------	------

argc

2

```
$ ./initials Kelly Ding
```

```
$ ./initials Kelly Ding
```

argv[1]

argv[2]

```
$ ./initials Kelly Ding
```

```
argv[1][0] argv[2][0]
```

Questions?



Initials Exercise

Write a program, `initials.c`, that takes a user's full name as input, and outputs their initials.

- Accept a user's name as command line arguments
- Assume that the user's names will be separated by one space

```
$ ./initials
Usage: ./initials First Last
$ ./initials David J. Malan
DJM
```

Addition Exercise

Write a program, `addition.c`, that adds two numbers provided as command-line arguments.

```
$ ./addition 2 8
2 + 8 = 10
$ ./addition 2
Usage: ./addition x y
```

This is CS50

Week 2